SECURITY · MIGRATION GUIDE

# PHP Commerce: Moving Off Shared Hosting

Familiar is not the same as safe. What to move, where to, and in what order — after the October 2026 delistings and checkout hijack.

---

> **Familiar is not the same as safe.** Protecting customers now takes wholesale change, not patches — especially where personal data (PII), card data (PCI) or health data (PHI) is involved. The October 2026 checkout hijack ran through apps merchants had trusted for years. Shopify and Google have been raising these requirements since 2022. Most of the deadlines have passed; what remains is enforcement — delisting, blocked installs and lost ad eligibility.

**Who this is for:** merchants, agencies and app developers running WordPress and WooCommerce, Magento or Adobe Commerce, or PHP-backed Shopify apps — loyalty, upsell, product options, wishlists — on shared or single-VM hosting.

**The question it answers:** what to move, where to, and in what order, so that one breached plugin, one compromised vendor or one runtime flaw cannot reach every customer on the server.

The PHP runtime itself is covered in the companion article: patch for **php2shell** (CVE-2026-109618 and CVE-2026-109619) and keep the Cloudflare WAF rules in Block — Entitlement Security → php2shell. This article is about the architecture around it. For the owner and legal view of the same deadlines — what goes dark, and who carries the liability — see the Risk & Liability Brief: The 2026 Client-Side Cliff.

# Vocabulary

| TERM | MEANS | IS NOT |
| --- | --- | --- |
| **Shared VM** | One server where many sites share an operating-system user, a database user and a filesystem | A private server with isolated apps |
| **Private VM with Docker** | One server the internet cannot connect to, with each application in its own sealed container, reached only through a Cloudflare Tunnel | Anything with an open inbound port |
| **TLS transport** | The sealed, tamper-evident envelope data travels in on every hop — shopper to edge, edge to server, server to Xano, server to database | Only the padlock in the browser |
| **Sidecar** | A small helper that runs beside each app and does one job — here, asking Xano `/auth/me` who the caller is and attaching a verified answer | The code-injection attack; it closes the hole where an attacker types someone else's customer ID |
| **Real-time data binding** | Prices, stock and points read live from the system that owns them | A copy refreshed by a nightly sync |
| **AI task binding** | An automated job limited to one tenant, one task and the tools that task needs | An agent holding a platform key |
| **Client-side redirect** | A script in the shopper's browser sending them to another domain | A 301 issued by the server |

# The requirements are not new

| SINCE | REQUIREMENT | WHAT IT DEMANDS | REFERENCE |
|---|---|---|---|
| Oct 1, 2022 | Shopify protected customer data (API 2022-10) | Personal data redacted by default; apps apply for access and meet Level 1 and Level 2 data-protection requirements | Shopify |
| Mar 2022, mandatory Mar 31, 2025 | PCI DSS v4.0, requirements 6.4.3 and 11.6.1 | An authorised inventory of every script on payment pages, integrity checks, and alerts on unauthorised change — the control that catches a checkout hijack | BDO summary |
| Mar 2024 | Google Consent Mode v2 for EEA traffic | Consent signals for ad storage, analytics, ad user data and ad personalisation | Google |
| Aug 26, 2024 | Shopify Customer Account API | At least Level 1 protected-customer-data approval | Shopify |
| Apr 1, 2025 | New public Shopify apps GraphQL-only; REST Admin API legacy | New App Store submissions cannot use REST | Shopify Partners |
| Jun 30, 2026 | Shopify Scripts stop executing | Checkout logic must be Shopify Functions | Shopify changelog |
| Oct 16, 2026, 15:00 UTC | Shopify API version 2025-10 no longer accessible | Apps still on unsupported resources after an upgrade deadline are delisted | Shopify versioning |

# October 2026: delistings and the checkout hijack

About **126 apps** were pulled from the Shopify App Store between October 14 and 18, 89 of them on October 15. Most of that number is versioning, not attack: API 2025-10 stopped being accessible on October 16, and delistings are routine — LetsMetrix counted 512 between September 22 and October 21, about 17 a day. Shopify has published no count and named no apps.

Only a few are tied to the hijack. Between October 10 and 20, attackers changed vendor-side settings so that shoppers clicking Checkout were sent to a lookalike payment page. **Shopify's own checkout was not breached.** The code came from vendors' configuration APIs and internal tools, not from the app files Shopify hosts — which is the risk of a third-party server holding store access.

| APP | STATUS | BASIS |
|---|---|---|
| Hulk Product Options | Hijack confirmed (Oct 10) | Developer statement — cside |
| Selleasy | Hijack confirmed (Oct 13) | Developer statement — LetsMetrix |
| Swym Wishlist Plus | Hijack confirmed (about Oct 14) | Unauthorised configuration change — LetsMetrix |
| Rebuy Engine — PHP app (PHP 8, CodeIgniter 4, per its job postings) | Delisted; **not** checkout hijacking, self-reported by the founder | Founder statement: on Oct 14, attackers found an unprotected sign-up path that skipped inbox verification, registered accounts on the company's own email domain, and inherited staff privileges on internal development stores. They read product settings and data-source rules for 11 merchant stores; no edits, no API keys or tokens exposed; merchants notified within 24 hours; contained within hours. Not independently verified |
| Privy; Boost AI Search & Filter | Delisted Oct 18; link to the hijack unconfirmed | Tracker listings, no vendor confirmation |
| 17TRACK; Consentmo GDPR Compliance | Delisted; link unconfirmed | Tracker listings; Consentmo's rests on a single confidential source |

Rebuy's account is the identity half of the same lesson. Nothing in the runtime failed: an email address on the company's domain was treated as proof of being staff, and staff accounts carried standing access to stores. Its remediation — privileges only for a named group, multi-factor authentication for every merchant, a third-party penetration test of the authentication services — is the permissions model this article argues for: identity verified at the server,

never inferred from what the caller typed, and access granted per task rather than held by default.

The same mistake is not unique to apps. Identity providers and the software built on them have shipped it too:

| CASE | WHAT WAS TRUSTED | WHAT IT ALLOWED | REFERENCE |
|---|---|---|---|
| **Microsoft Entra ID — nOAuth** (first disclosed June 2023; still present in about 1 in 10 of 100+ tested SaaS apps in June 2025) | An unverified email claim used as the user's identifier | With only an attacker-controlled Entra tenant and the victim's email address, full takeover of the victim's SaaS account. MFA, conditional access and Zero Trust policies did not stop it | Semperis |
| **Okta AD/LDAP delegated authentication** (advisory Nov 1, 2024; introduced July 2024) | A cached earlier login for usernames longer than 52 characters — often email addresses | Sign-in with the username alone, without the right password, where MFA was off | Okta advisory |
| **Rebuy Engine** (Oct 14, 2026, founder-reported) | An email address on the company's own domain, never verified | Staff-level reads on internal stores and settings reads on 11 merchant stores | Founder statement |

The rule all three break is the same: **an email address is a contact detail, not an identity.** Identify a user by the identity provider's immutable subject — the OpenID Connect `sub` with its issuer, in Entra the object ID with the tenant ID — require a verified email before linking any account, restrict which tenants may sign in, and grant elevated access only to a named group. MFA is necessary and, as nOAuth shows, not sufficient.

**Merchant checks:** find these apps among those installed, delisted ones included; review app embeds and custom JavaScript fields for scripts nobody added; run a test checkout in a private window and confirm it lands on Shopify's checkout domain; read Shopify's notice for your store's exposure window. Shoppers who paid by Shop Pay or PayPal were generally not exposed; shoppers who typed a card number during a window were.

**Remove every client-side redirect to another domain**

The hijack worked because a script in the shopper's browser could send them somewhere else. Any script that can do that today is a hijack waiting for a compromised vendor — legitimate ones included: market redirectors, affiliate rewriters, "faster checkout" buttons, leftovers from a previous storefront.

| REMOVE — CLIENT SIDE | REPLACE WITH — SERVER SIDE |
|---|---|
| `window.location` or `location.href` in theme code, app embeds or tag managers | Redirect rules at the Cloudflare edge or a Worker: a 301 to an allow-listed destination |
| `<meta http-equiv="refresh">` pointing off-domain | The same edge redirect, or a link the shopper chooses |
| Scripts that rewrite links or the checkout button | Checkout posts only to the store's own Shopify checkout |
| Geo or market redirectors in the browser | Market routing at the edge, on Shopify Markets domains or subfolders |
| Old domains forwarded by script | Domain-level 301s in one redirect table under version control |

Then hold it: a destination allow list for every server redirect; a Content Security Policy with `script-src` limited to an inventoried list and `form-action` limited to the store's own origins and checkout; a payment-page script inventory with change alerts (PCI DSS 6.4.3, 11.6.1); and a private-window test checkout on every release.

# What is being replaced

| RISK ON A MAMP/WAMP-STYLE SHARED VM | WHAT HAPPENS | WHAT REPLACES IT |
|---|---|---|
| One OS user for every site | A file-read or upload bug in one plugin reads every site's `wp-config.php` or `env.php` | One container per site, read-only filesystem, no shared volumes |
| Public origin IP | Attackers skip the CDN and WAF | Origin reachable only through a Cloudflare Tunnel |
| TLS ends on the box, or not at all | Plain HTTP between CDN and origin | Edge certificate, Full (strict), origin certificate or tunnel |
| One database user | Every app can read every table | A role per service, plus Postgres row-level security |
| Admin panels on the open web | `/wp-admin`, Magento admin and phpMyAdmin face everyone | Cloudflare Access on every admin path |
| Frozen PHP versions | End-of-life PHP survives because upgrading breaks a neighbour | A pinned image per site, rebuilt and scanned each release |
| Secrets in files | Keys inside the web root | Secrets injected at runtime |
| No audit trail | FTP edits leave no record | Git commit, image digest and deploy tag per change |

# The options, and what each one isolates

| OPTION | TLS AND ORIGIN | CODE ISOLATION | ADMIN LOGIN | DATA ISOLATION | SCALING |
|---|---|---|---|---|---|
| Shared VM (today) | TLS on the box or none; public IP | None | Passwords on the open web | One database user | Up only |
| WordPress or Magento in Docker behind Cloudflare | Edge TLS, Full (strict); tunnel; no inbound ports | Container per app | Cloudflare Access | Role per service | One VM, up |
| EmDash on Cloudflare Workers | Edge TLS; no origin | Isolate per plugin, permissions manifest | Passkeys; SSO | Plugins have no direct database access | Pay per use |
| 11ty + Decap CMS on Cloudflare | Edge TLS; no origin | No code runs per request | Access on `/admin` + GitHub OAuth | Static files; live data through APIs | Static requests free and unlimited |
| Jekyll on GitHub Pages | GitHub-managed TLS | No code runs per request | GitHub accounts | Static files | Free within 1 GB and a 100 GB/month soft limit; **not permitted for e-commerce** |
| Shopify Functions in Rust | Runs inside Shopify | WebAssembly sandbox, declared input only | Shopify app permissions | Scoped to the shop | Shopify scales it |
| Kubernetes + Postgres RLS | Edge TLS; NetworkPolicy; TLS to the database | Pod and namespace | Access + SSO | Row-level security per tenant and actor | Horizontal; highest cost |

**Order of moves:** static first, for every page that needs no login. EmDash next, where editors need a live CMS. WordPress or Magento in Docker last, only for what cannot yet leave PHP.

**EmDash** was announced by Cloudflare on April 1, 2026 as a v0.1.0 developer preview — MIT licensed, TypeScript on Astro, no WordPress code. Content imports from a WXR export or the EmDash Exporter plugin; themes must be ported. It has no commerce, so WooCommerce stays in Docker or the store moves to Shopify. Pilot it on a blog subdomain first. Cloudflare: EmDash

**The static path** has run in production for years — Jekyll and GitHub Pages since 2008, 11ty since 2017, Decap since 2016 as Netlify CMS. GitHub Pages' terms forbid running an online business or e-commerce site on it, so a store built with Jekyll or 11ty is hosted on Cloudflare.

## Shopify plugins become Rust Shopify Functions

| OLD PLUGIN PATTERN | REPLACE WITH | WHY |
|---|---|---|
| Line-item Script — tiers, BOGO, loyalty redemption | Discount Function | Runs inside Shopify on declared input; no merchant server to breach |
| Shipping Script | Delivery customization Function | Same |
| Payment Script | Payment customization Function | Same |
| Upsell or bundle app editing the cart | Cart transform Function + theme app extension | No third-party server holding order scopes |
| checkout.liquid fields and validation | Cart and checkout validation Function + Checkout UI extension | No arbitrary script on the payment page |
| Thank-you page script tag for points or pixels | HMAC-verified order webhook to a ledger Worker, plus a consent-aware web pixel | Events verified; tracking honours consent |
| PHP app server on the REST Admin API | GraphQL Admin API from a Worker | Meets the GraphQL-only rule; no PHP host |

Shopify recommends Rust for public apps, large carts and complex logic, because it compiles directly to WebAssembly and stays inside the instruction-count limits; JavaScript is for prototypes. Functions decide; they do not store. Points, history and consent stay in the ledger, behind `/auth/me`.

## Identity, data and AI tasks

1. **Identity.** A sidecar resolves every request through Xano `/auth/me` with the caller's JWE token and forwards the actor in a signed internal header. PHP never parses tokens and never trusts a `user_id` sent in a form.
2. **Data.** Postgres row-level security filters every read and write by tenant and actor, so a plugin bug or SQL injection returns only the caller's own rows. MySQL has no RLS; WooCommerce and Magento data either moves or sits behind filtered views.
3. **AI tasks.** Each automated job runs in a Tool Runner loop with a scoped tenant key and only the tools that task needs — the runner exists in PHP too. The full model — Cloudflare binds, Xano grants, Rust checks — is in Entitlement Security → AI task binding.

| HOP | PERMISSIONS | TLS TRANSPORT |
| --- | --- | --- |
| Browser or agent → edge | Access policy, or the Xano JWE | TLS 1.2 minimum; HSTS |
| Edge → origin | Origin accepts only Cloudflare | Tunnel; no inbound ports |
| Runner → tools | Scoped tenant key, per-task tool list | HTTPS; Access service tokens |
| Worker → Xano | Workspace-scoped key; permission check in Xano | HTTPS; a shared-secret header so the direct Xano URL refuses traffic that skipped the Worker |
| Shopify → store logic | Function input query; webhook HMAC, constant-time compare | Shopify-hosted; HTTPS webhooks |
| Service → database | Role per service; actor and tenant set per request | `sslmode=verify-full` |

# Google Merchant API: rebuild loyalty, reviews and comparison in one pass

The Google feed integration is being rebuilt regardless. Merchant API became generally available as v1 in July 2025, its v1beta versions were discontinued on February 28, 2026, new features ship only there, and conversion sources move from `content/v2.1` to `conversions/v1`. Connect loyalty, reviews and price comparison directly in the same conversion, rather than through a plugin that can be delisted.

| GOOGLE SURFACE | MERCHANT API | NEEDS | REFERENCE |
|---|---|---|---|
| Loyalty — member prices, points, loyalty shipping | `loyaltyPrograms[]` on product attributes | A program linked in Merchant Center; AU, JP, UK, DE, FR, US; member prices matching feed, landing page and checkout | Product attributes · Attribute spec |
| Product reviews | Reviews sub-API, `productReviews` | Product ratings enrolment; `is_verified_purchase` set from a real order | Guide |
| Seller reviews | Reviews sub-API, `merchantReviews` | Store ratings enrolment | Guide |
| Price comparison | Reports sub-API — price competitiveness, price insights, competitive visibility | Standalone or individual sub-account | Reports |
| Comparison Shopping Services | CSS API; `Csses` folds into Accounts | A CSS partner | CSS |
| Conversions | Conversions sub-API, `conversionSources` | Field renames: `property`, `destination`, `report` | Migrate |

A verified-purchase review or a member price shown to Google is a claim about a real order and a real member. CRM Sync records that claim as a signed mandate anyone can check offline in a browser — verify a mandate · verified transactions.

## Supply chain: every new ecosystem is a new door

Moving off PHP brings in Python for AI gateways, npm for Workers and Rust crates for Shopify Functions. On March 24, 2026, LiteLLM 1.82.7 and 1.82.8 were published to PyPI with a credential stealer — one through a `.pth` file that runs on every Python start — taking SSH keys, cloud and Kubernetes credentials, `.env` files and TLS private keys (BleepingComputer). A second

poisoned release, 1.107.0, followed on October 21 — Entitlement Security → LiteLLM.

| ECOSYSTEM | PERMISSIONS CHECK BEFORE INSTALL | WHAT IT CATCHES |
|---|---|---|
| Rust crates | `cargo vet --locked` in CI: every version covered by a trusted published audit, or reviewed and certified | A hijacked release nobody has reported yet |
| Python | Hash-pinned installs; versions at least two weeks old | A new malicious release |
| npm | Committed lockfile; `npm ci --ignore-scripts`; an osv-scanner baseline with expiry dates | Install-time scripts; known-bad versions |
| GitHub Actions | Pinned to commit SHAs | A retagged action |

CRM Sync's desktop build imports the published crate audits of Mozilla, Google, the Bytecode Alliance, ISRG, Embark and Zcash — 102 crates fully audited, 357 recorded as a starting baseline — and fails CI on any crate version no audit covers. Wildcard trust is avoided on purpose: a hijacked publisher would pass it.

# What it costs

| COST | WHAT IT MEANS | SEVERITY |
|---|---|---|
| Rework of vendor plugins | Loyalty, upsell and options apps are rebuilt as Functions, Workers and a ledger, not configured | High — weeks per plugin, not hours |
| A monthly bill above shared hosting | A private VM, Cloudflare plans or managed Kubernetes cost more than a shared box | Low to high by tier; get current quotes |
| EmDash maturity | A v0.1.0 preview with a near-empty plugin ecosystem; WordPress's co-founder argues the isolation only holds on Cloudflare | Medium — pilot only |
| MySQL has no row-level security | WooCommerce and Magento data needs a move to Postgres or filtered views | Medium |
| New dependency ecosystems | Rust, Python and npm each need their own gate | Medium — the gates are the cost |
| Familiar tooling retired | Teams lose cPanel, FTP and plugin-store installs | Real, and the point |

> **What this does not claim.** It is not a certification and not legal advice. Delisting counts come from public trackers, not Shopify. "Unconfirmed" means exactly that. Prices are deliberately absent: they vary by provider, region and traffic.

# What to do next

Ordered by exposure — card data first.

1. **Payment pages.** Inventory every script, remove every client-side redirect, set the CSP, run a private-window test checkout. *Owner: store owner with the developer.*

2. **Delisted and hijack-listed apps.** Remove or replace; revoke their Shopify scopes after cutover. *Owner: store owner.*
3. **PHP runtime.** Patch for php2shell and keep the WAF rules in Block. *Owner: hosting or DevOps.*
4. **Origin.** Cloudflare in front, Full (strict), Access on every admin path, tunnel, inbound ports closed. *Owner: DevOps.*
5. **Containers.** One pinned image per app, secrets injected at runtime. *Owner: developer.*
6. **Identity and data.** `/auth/me` sidecar; a database role per service; RLS or filtered views; personal fields encrypted with a key generated for this deployment. *Owner: developer with compliance.*
7. **Checkout logic.** Scripts and checkout.liquid patterns rebuilt as Rust Shopify Functions. *Owner: Shopify developer.*
8. **Google.** Merchant API migration with loyalty, reviews, comparison and conversions in the same pass. *Owner: marketing operations with the developer.*
9. **Content.** Static publish first, EmDash pilot second. *Owner: content lead.*
0. **Decommission** the shared VM after the tunnel has carried production traffic for a full billing cycle. *Owner: DevOps.*

---

*Evidence basis: public vendor, platform and standards documentation linked inline, read on 22 October 2026. Not a certification.*

---